Adaptive Stability-Aware Scheduling for Cloud-Orchestrated Volunteer Computing Using Heterogeneous Desktop and Laptop Resources

Mishti Parikshit Gandhi, Alan James, and Abhyuday Thapliyal
Vellore Institute of Technology, Vellore, India

***Abstract*—Volunteer computing can harvest substantial computational power from idle desktop and laptop computers, but volunteer devices become unavailable because of user activity, power changes, battery conditions, network failures, and correlated outages. This paper presents an adaptive scheduler that re-estimates the interruption risk of each running task and adds protection only when that risk becomes high. Instead of replicating every task from the beginning, it checkpoints the task and starts a secondary execution from that checkpoint on a volunteer device whose availability history is weakly correlated with the primary, and it falls back to cloud execution when risk is high. Risk is computed from a state-dependent conditional survival probability over the task's remaining execution time. We evaluate the scheduler in a discrete-time simulation of heterogeneous desktops and laptops with individual and correlated interruptions, using 300 random seeds per configuration and an open-source simulator. Under high instability, the scheduler completes 99.8% of tasks by their deadline, against 90.3–93.4% for volunteer-only baselines. When the baselines are also allowed to restart lost tasks in the cloud, they match this deadline completion, but the proposed scheduler needs 46–71% less cloud time across instability levels. With hedging thresholds chosen on a separate set of seeds, it also wastes no more volunteer CPU than the best baseline. An ablation study identifies the live risk controller as the decisive component, and shows that the state-dependent hazard multipliers did not help in our environment.**

***Index Terms*—volunteer computing, adaptive scheduling, checkpointing, replication, survival analysis, correlated failures, cloud fallback**

I. Introduction

Volunteer computing runs workloads on resources contributed by the public. Platforms such as BOINC [1] have supported long-running science projects for years, and during the COVID-19 pandemic the Folding@home network exceeded an exaflop of aggregate throughput [2]. Desktops and laptops are an attractive source of such capacity, but their availability differs from that of managed servers: owners interact with the machine, disconnect it, switch to battery power, or shut it down, often without warning.

These events create a scheduling problem that differs from conventional resource allocation. A task may be placed on a suitable device and still become unreliable later in its execution, so reliability cannot be treated as a fixed property evaluated only at dispatch time. Replicating every task across several devices is a common remedy, but copies started from the beginning waste computation whenever the primary succeeds and add network, storage, and scheduling overhead. A more selective strategy lets a task run normally while its interruption risk is low and adds redundancy only when that risk rises. The central idea of this work is therefore:

> *When a running volunteer task is likely to be interrupted, capture its current state and start a redundant execution from that checkpoint on a resource whose availability is unlikely to be correlated with that of the primary.*

The individual mechanisms we use (checkpointing, replication, failure prediction, correlation-aware placement, and cloud bursting) are each well studied. Our contribution is to combine them into a single risk-driven control loop for volunteer resources and to evaluate that loop systematically. Specifically, we contribute:

- a live interruption-risk estimate, based on a state-adjusted conditional survival probability over the task's remaining execution time, that is updated throughout execution;
- a four-level execution policy (normal, checkpointed, checkpoint-seeded hedge, cloud fallback) driven by that estimate;
- a shrinkage estimator of joint interruption probability that combines empirical co-availability with an independence prior when selecting hedge devices; and
- a simulation study across instability levels, risk thresholds, and pool sizes, including an ablation that isolates each mechanism, a comparison against baselines given equal cloud access, and an out-of-sample check of the chosen thresholds.

Our main finding is that, in a hybrid volunteer–cloud system, the scheduler reaches the deadline completion of a simple "restart in the cloud" policy with a third to a half of its cloud time.

The rest of the paper is organised as follows. Section II reviews related work. Section III defines the system model. Section IV describes the scheduler. Section V gives the experimental methodology, Section VI the results, and Section VII discusses limitations. Section VIII concludes.

II. Related Work

A. Scheduling on volunteer resources

BOINC [1] is the dominant volunteer-computing middleware; it relies on static replication for both fault tolerance

and result validation. Anderson [3] studies global scheduling across volunteer machines with different capabilities, which motivates our first-stage capability filter before reliability-based ranking. CLARA [4] groups nodes with complementary uptime patterns, showing that availability should be evaluated over a *set* of resources rather than node by node; our co-availability score adapts this idea to the choice of a single hedge device for a task that is already running. NARA [5] adds network location, latency, and energy to volunteer placement, which motivates the network term in our suitability score. Other work extends volunteer computing to vehicular networks [6], mobile devices [7], and volunteer edge-cloud deployments in which trust and reliability of contributed nodes drive allocation [8]. Heterogeneity-aware task scheduling has also been studied for distributed parallel systems in general [9] and for cloud environments [10].

B. Checkpointing, replication, and speculative execution

Fault-tolerance surveys distinguish reactive mechanisms such as checkpoint/restart and replication from proactive monitoring, prediction, and migration [11]–[13], and note the resource and energy overhead that each mechanism imposes [14], [15]. Setlur et al. [16] combine replication heuristics with checkpointing for cloud workflows, and Khaleel [17] provisions resources for edge offloading with failure awareness. For pre-emptible cloud capacity, Tomar et al. [18] adapt checkpointing to spot instances, and Teylo et al. [19] schedule deadline-constrained bag-of-tasks applications on hibernation-prone spot VMs, migrating work to on-demand VMs when deadlines are at risk; our cloud fallback plays the same role for volunteer pools. Hedging is closely related to speculative execution for stragglers, surveyed by Gill et al. [20]; START [21] predicts stragglers with an encoder LSTM and mitigates them before they delay a job. Our hedge differs in that it starts from a checkpoint instead of from the beginning and is triggered by predicted *interruption* rather than slowness.

C. Failure prediction and correlated failures

Failure-prediction models use operational telemetry to anticipate task, job, and node failures [22]–[24], although missing or incomplete telemetry degrades their accuracy [25]. Survival analysis models time-to-failure directly and handles censored observations naturally [26]; we adopt this view and express interruption risk as a conditional survival probability with explicit state-dependent hazard multipliers rather than a learned classifier. When replicas share a failure cause, independence assumptions underestimate joint failure; Mills et al. [27] give placement algorithms for correlated failures in hierarchical failure domains. Our co-availability estimator addresses the same issue without requiring a known failure-domain hierarchy.

D. Gap

To our knowledge, no prior work evaluates, for volunteer desktops and laptops, the combination of live risk re-evaluation, checkpoint-seeded hedging, correlation-aware hedge placement, and cloud fallback, with both deadline completion and wasted computation as outcome measures. This paper provides that evaluation.

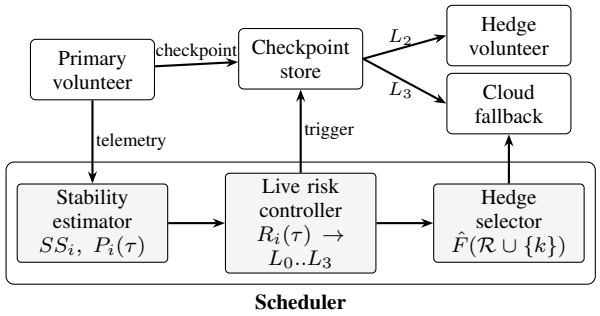


Fig. 1. Scheduler architecture. The live risk controller re-evaluates each running task. When risk crosses R_2 the task is checkpointed and a hedge resumes from the checkpoint on a volunteer chosen for low co-availability with the primary (L_2); when risk reaches R_3, a copy resumes in the cloud (L_3).

TABLE I
NOTATION

Symbol	Meaning
C_i, M_i, W_i	CPU capacity, memory, bandwidth of device i
π_i, B_i, H_i	power state, battery state, availability history
SS_i	stability score of device i
A_i, U_i, Q_i, Y_i	availability, utilisation, network-quality and historical-stability factors, each in $[0, 1]$
$\omega_a, \omega_u, \omega_q, \omega_y$	stability-score weights, summing to 1
a_i	age of the current availability session of device i
τ	remaining execution time of the task on device i
$P_i(\tau), R_i(\tau)$	survival probability and interruption risk
$\lambda_i(t), m_k$	baseline hazard and state multiplier for factor k
Φ_{ij}	suitability of device i for task j
R_1, R_2, R_3	risk thresholds for execution levels L_1, L_2, L_3
$x_i(w)$	1 if device i was available in history window w
$\hat{F}(\mathcal{R})$	estimated joint interruption probability of set $\mathcal{R}$

III. System Model

The system consists of a scheduler, a pool of heterogeneous volunteer devices, a checkpoint store, and an optional cloud execution service (Fig. 1). Table I summarises the notation.

A. Devices and tasks

Each volunteer device i is described by

$$D_i = (C_i, M_i, W_i, \pi_i, B_i, H_i), \tag{1}$$

where C_i is computational capacity, M_i available memory, W_i network bandwidth, π_i the current power state (AC or battery), B_i the battery state, and H_i the historical availability record. A task j has computational demand, memory demand, network requirements, an estimated execution time E_j, and a deadline d_j. Tasks are independent; workflows with dependencies are outside the scope of this paper.

B. Stability score

The scheduler maintains a continuously updated stability score

$$SS_i = \omega_a A_i + \omega_u U_i + \omega_q Q_i + \omega_y Y_i, \qquad \sum \omega = 1, \quad (2)$$

where A_i, U_i, Q_i, and Y_i are availability, utilisation, network-quality, and historical-stability factors, each in $[0, 1]$ with larger values indicating a more dependable device. Factors are updated by exponential smoothing,

$$X_i^{\text{new}} = \alpha X_i^{\text{recent}} + (1 - \alpha) X_i^{\text{history}}, \qquad 0 < \alpha \leq 1, \quad (3)$$

so that old observations do not dominate current device conditions. In our implementation, Y_i is the fraction of history windows in which device i was available; A_i starts as that fraction over the most recent week and is then updated every hour by the smoothing rule with $X_i^{\text{recent}} \in \{0, 1\}$ the device's current availability. The simulator does not model utilisation or link quality per device, so U_i and Q_i take the same values (0.5 and 0.8) for all devices; they shift every score equally and do not affect rankings, which are therefore driven by A_i and Y_i.

IV. Proposed Scheduler

A. Interruption risk from conditional survival

Reliability is evaluated over the task's *remaining* execution period rather than over the device's whole history. Let T_i be the time until the next interruption of device i, and let z_i denote its currently observed state (session age, power and battery state, recent network behaviour). For a running task, τ denotes its expected remaining execution time on device i, which shrinks as the task makes progress. The conditional survival probability and the corresponding interruption risk are

$$P_i(\tau) = \Pr(T_i \geq \tau \mid z_i), \qquad R_i(\tau) = 1 - P_i(\tau). \quad (4)$$

A device with good long-term reliability can therefore still receive a high risk score when its current state indicates that it may soon become unavailable, and the same device yields a lower risk for a task that is nearly finished than for one that has just started.

B. State-aware hazard adjustment

Historical survival estimates cannot capture short-term changes in behaviour; a laptop on battery, for example, is more likely to be interrupted than the same laptop on AC power. We therefore scale the baseline hazard $\lambda_i(t)$, expressed as a function of session age, by multiplicative factors for each active state condition $k \in \mathcal{K}_i$:

$$\lambda_i^{\text{adj}}(t) = \lambda_i(t) \prod_{k \in \mathcal{K}_i} m_k. \quad (5)$$

With a_i the current session age, the adjusted survival probability is

$$P_i(\tau) = \exp\left(- \int_{a_i}^{a_i + \tau} \lambda_i^{\text{adj}}(u)\, du \right), \quad (6)$$

which is used in (4). This is a proportional-hazards form. In our implementation the hazard is held constant over the remaining horizon, $\lambda_i^{\text{adj}} = (1/\text{MTBF}_i)\, g(a_i) \prod_k m_k$ with session-age factor $g(a_i) = 1 + \min\{1, a_i/(2\,\text{MTBF}_i)\}$, so that (6) reduces to $P_i(\tau) = \exp(-\lambda_i^{\text{adj}} \tau)$. The state factors are battery power, low battery ($< 20\%$), and an unstable network link (a current fluctuation or an uplink below 5 Mbps); their multipliers are listed in Table II. The remaining time is $\tau = (1 - q) E_j / v_i$, where q is the task's completed fraction, E_j its work in steps on a 3-core reference device, and v_i the device's speed factor.

The age factor and the multipliers encode the design's assumptions that interruption risk grows with session length and that battery and network problems make interruption more likely. Their values are set by assumption, not fitted to data; we keep them as specified so that the scheduler is evaluated as designed. Our simulated environment does not share all of these assumptions: individual outages are memoryless, a laptop on battery is interrupted only when its battery is exhausted, and network fluctuations cause no interruptions. The ablation (Section VI) therefore also tests whether a mis-specified risk model still helps.

C. Task–device suitability

Reliability alone is insufficient: a stable device may lack the CPU, memory, or bandwidth a task needs. After filtering out devices that cannot meet a task's minimum requirements, the scheduler ranks the remaining candidates by

$$\begin{aligned} \Phi_{ij} = {} & w_c C_{ij} + w_m M_{ij} + w_n W_{ij} + w_s SS_i \\ & + w_p P_i(\tau_{ij}) + w_d \Delta_{ij} - w_k K_{ij}, \end{aligned} \quad (7)$$

where every term lies in $[0, 1]$. The match terms are capped ratios, $C_{ij} = \min\{1, c_i/c_j\}$, $M_{ij} = \min\{1, m_i/m_j\}$, and $W_{ij} = \min\{1, b_i/b_j\}$, of the device's cores, memory, and uplink to the task's requirements. The deadline term $\Delta_{ij} = \min\{1, (d_j - t)/\tau_{ij}\}$ compares the time left before the deadline with the remaining execution time on device i. The cost term K_{ij} is 0.10 for devices with an uplink below 10 Mbps and 0.02 otherwise, reflecting checkpoint-transfer cost. The weights (Table II) on the six positive terms sum to 1, the cost weight is 0.05, and all are set by assumption.

D. Execution levels and the live risk controller

Each running task is assigned one of four execution levels using thresholds $0 < R_1 < R_2 < R_3 < 1$:

$$L = \begin{cases} L_0 \text{ (normal)}, & R < R_1, \\ L_1 \text{ (checkpointed)}, & R_1 \leq R < R_2, \\ L_2 \text{ (checkpoint-seeded hedge)}, & R_2 \leq R < R_3, \\ L_3 \text{ (cloud fallback)}, & R \geq R_3. \end{cases} \quad (8)$$

The key difference from static replication is that the level is not fixed at dispatch. The controller periodically re-reads each primary device's state, re-estimates τ, recomputes $R_i(\tau)$, and moves the task between levels (Algorithm 1). Once any execution returns a valid result, the remaining copies are cancelled.

Algorithm 1 Live risk control for an active task j on device i

Observe current state z_i; estimate remaining time τ
Compute $R_i(\tau)$ using (6)
if $R_i(\tau) < R_1$ **then**
    Continue normal execution ▷ L_0
else if $R_i(\tau) < R_2$ **then**
    Maintain periodic checkpoints ▷ L_1
else if $R_i(\tau) < R_3$ **then**
    Checkpoint; $k \leftarrow$ SELECTHEDGE($\{i\}, \mathcal{C}$) ▷ L_2
    Resume a hedge of j on k from the checkpoint
else
    Checkpoint ▷ L_3
    Resume a copy of j in the cloud from the checkpoint
end if
On first completed result: accept it and cancel other copies

E. Checkpoint-seeded hedging

Conventional replication starts every copy from the beginning. Instead, when the primary becomes risky after completing fraction q of the work, the scheduler captures a checkpoint and resumes a secondary execution from it, so both copies proceed from q toward completion and the work done before the hedge was launched is not repeated. A checkpoint contains the task identifier, a sequence number, a completed-work indicator, the application state, integrity information, and a timestamp. In the simulation, checkpoints are 100 MB, a size chosen by assumption rather than measured for a specific application. Moving a checkpoint takes $800/b$ seconds over a link of b Mbps (8 s at 100 Mbps, 80 s at 10 Mbps, 800 s at 1 Mbps), during which the affected copy makes no progress. Uploading delays the primary each time it checkpoints; downloading delays the start of a hedge; and a cloud copy is assumed to receive the checkpoint at 100 Mbps. A copy resumed from a checkpoint continues from the saved fraction q; progress made after the last checkpoint is lost if its device fails.

F. Co-availability-aware hedge selection

The most reliable individual device does not necessarily form the most reliable *pair* with the primary: devices on the same site, network, power circuit, or user schedule tend to fail together. Each device's history is therefore stored as an availability bitmap, with $x_i(w) = 1$ if device i was available in window w and 0 otherwise. For a set $\mathcal{R}$ of devices executing the same task, the empirical probability that all of them are unavailable in a window is

$$F_{\text{emp}}(\mathcal{R}) = \frac{1}{N} \sum_{w=1}^{N} \prod_{i \in \mathcal{R}} \big(1 - x_i(w)\big), \tag{9}$$

where N is the number of history windows. Assuming independence instead gives

$$F_{\text{ind}}(\mathcal{R}) = \prod_{i \in \mathcal{R}} (1 - A_i), \tag{10}$$

Algorithm 2 SELECTHEDGE($\mathcal{R}, \mathcal{C}$)

Remove from $\mathcal{C}$ devices that fail the task's requirements
for each $k \in \mathcal{C}$ **do**
    $J_k \leftarrow \hat{F}(\mathcal{R} \cup \{k\}) + \beta K_k$
end for
return $\arg\min_k J_k$ (or NONE if $\mathcal{C} = \emptyset$)

where A_i is the recent availability of device i, so $1 - A_i$ is its probability of being unavailable in a window. Because F_{emp} is noisy when N is small or joint outages are rare, we shrink it toward the independence estimate:

$$\hat{F}(\mathcal{R}) = \frac{N F_{\text{emp}}(\mathcal{R}) + s F_{\text{ind}}(\mathcal{R})}{N + s}, \tag{11}$$

with prior strength s. Correlated interruptions enter through the bitmaps themselves: devices that share an outage cause are unavailable in the same windows, so F_{emp} is larger for such a pair than the independence estimate predicts, and the pair is avoided. With $s = 50$ and $N = 8{,}064$ five-minute windows of history, the independence estimate receives under 1% of the weight; it matters only when a history is short. The value of s, like α, is set by assumption and was not tuned. Algorithm 2 selects the hedge

$$k^\star = \arg\min_{k \in \mathcal{C}} \ \hat{F}(\mathcal{R} \cup \{k\}) + \beta K_k, \tag{12}$$

where $\mathcal{C}$ is the set of capable candidates, K_k the cost of using device k (we use the inverse of its uplink bandwidth, a proxy for checkpoint-transfer cost), and β a trade-off weight.

G. Cloud fallback and overall lifecycle

When the primary's risk reaches R_3, or when every volunteer copy of a task has been lost, the task resumes in the cloud from its latest checkpoint rather than restarting. The cloud is a fallback rather than the default, which preserves the cost advantage of volunteer resources under normal conditions. The complete lifecycle is: (i) collect capability and availability information; (ii) estimate stability; (iii) select and dispatch to a primary device; (iv) periodically update risk; (v) checkpoint in the protection region; (vi) launch a co-availability-aware hedge at high risk; (vii) fall back to the cloud if necessary; and (viii) accept the first completed result and cancel the remaining copies. The design balances three competing objectives: deadline reliability, wasted volunteer computation, and cloud cost.

V. EXPERIMENTAL METHODOLOGY

A. Simulation setup

We use a discrete-time simulator with a five-minute step. For each seed, four weeks of availability history are generated to estimate device behaviour; tasks then arrive over a fresh evaluation week, and the run continues until the last task's deadline has passed. Each device alternates between available and unavailable periods; its mean time between failures (MTBF) is drawn from an exponential distribution whose mean depends on the instability condition, and outages last

TABLE II
SIMULATION AND SCHEDULER PARAMETERS

Parameter	Value
Time step	5 min
History / arrival window	4 weeks / 1 week (run continues to last deadline)
Devices (main / scaling)	30 / 10–50
Device mix	desktop or laptop with equal probability
CPU cores (speed factor)	2 (0.75), 3 (1.0), or 5 (1.5)
Memory / uplink	4, 8, 16 GB / 1, 10, 100 Mbps
Device MTBF (mean of exp. draw)	650 / 200 / 75 steps (stable / moderate / high), minimum 15
Device outage duration	exponential, mean 12 steps
Correlation groups	4; group MTBF 2.5× / 2× / 1.5× device MTBF; outage mean 18 steps
Laptop power	starts on battery w.p. 0.4; switches w.p. 0.02/step; drains 2%/step, recharges 5%/step
Task arrivals	Poisson, 0.075 tasks/step (≈151 per week)
Task work	uniform 12–48 steps on a 3-core device; cloud runs at the 3-core rate
Task requirements	2–3 cores, 4–8 GB, 1–10 Mbps
Deadline factor	uniform 1.6–3.0 × expected time
Hazard multipliers m_k	battery 2.5, low battery (<20%) 4.0, unstable network 1.8
Smoothing factor α	0.30 (stability updated hourly)
Stability weights $\omega_{a,u,q,y}$	0.35, 0.25, 0.15, 0.25
Suitability weights $w_{c,m,n,s,p,d,k}$	0.25, 0.20, 0.15, 0.15, 0.15, 0.10, 0.05
Thresholds R_1, R_2, R_3	0.05, 0.20, 0.50
Checkpointing at L_1	every 15% of progress
Checkpoint size / transfer	100 MB; 800/uplink s
Shrinkage s / cost weight β	50 / 0.01
Seeds per configuration	300 (500 for Table VII)

one hour on average. Devices are split into four correlation groups, and each group suffers shared outages that make all of its members unavailable at once. Laptops switch between AC and battery power; a laptop whose battery runs out shuts down until it is plugged in again. The same processes generate the history and the evaluation week. Task arrivals follow a Poisson process (about 151 tasks per week), and each deadline is drawn between 1.6 and 3.0 times the task's expected execution time. A device runs one task replica at a time, so tasks wait when no capable device is free. Unless stated otherwise, the main configuration uses 30 devices. Table II lists all parameters; the simulator, its configuration, and an audit log are available in the project repository.

B. Baselines

We compare four strategies. *Capability only* places each task on the capable device with the most cores (then bandwidth and memory), without redundancy. *Capability + stability* places it on the capable device with the highest stability score SS_i. *Independence replication* is static two-way replication: at dispatch it starts the task on the two capable devices with the highest recent availability, without considering correlation, and keeps both copies running until one finishes. *Proposed* is the full scheduler of Section IV. When every copy of a baseline task is lost, the task is re-dispatched on a free volunteer and restarts from the beginning, because baselines do not checkpoint. In the main comparison only the proposed scheduler can use the cloud, as in its design. Because that gives it an extra resource, we also run an *equal-cloud* comparison in which a baseline task whose volunteer copies are all lost is restarted in the cloud instead of on another volunteer, through the same recovery path the proposed scheduler uses.

TABLE III
MAIN RESULTS (MEAN ± 95% CI; 30 DEVICES, 300 SEEDS). WASTED CPU IN 10^3 DEVICE-S, CLOUD IN 10^3 INSTANCE-S.

Cond.	Method	Deadline (%)	Wasted CPU	Cloud
Stable	Capability only	99.84 ± 0.04	87.0 ± 5.3	0
	Cap. + stability	99.18 ± 0.10	84.8 ± 4.6	0
	Indep. replication	99.79 ± 0.04	1003.7 ± 13.0	0
	Proposed	99.85 ± 0.03	31.6 ± 2.1	62.5 ± 3.1
	Proposed, tuned	99.86 ± 0.03	26.1 ± 1.6	65.2 ± 3.3
Moderate	Capability only	98.94 ± 0.16	208.1 ± 10.1	0
	Cap. + stability	97.56 ± 0.19	211.2 ± 9.0	0
	Indep. replication	98.48 ± 0.15	949.5 ± 12.6	0
	Proposed	99.79 ± 0.04	162.0 ± 10.7	145.8 ± 5.8
	Proposed, tuned	99.78 ± 0.05	58.5 ± 3.6	154.9 ± 5.6
High	Capability only	93.37 ± 0.45	548.2 ± 19.6	0
	Cap. + stability	90.30 ± 0.49	596.0 ± 19.5	0
	Indep. replication	91.84 ± 0.42	1067.7 ± 17.9	0
	Proposed	99.79 ± 0.05	556.7 ± 22.0	384.3 ± 14.0
	Proposed, tuned	99.71 ± 0.06	212.8 ± 12.2	363.3 ± 10.0

C. Metrics and statistics

We report deadline completion (percentage of tasks completed by their deadline); wasted CPU time; cloud consumption (instance-seconds); and replica launches per task beyond the primary. Wasted CPU is volunteer device time whose work is discarded, in device-seconds: the work of a copy that is cancelled or interrupted, except progress it saved in a checkpoint that another execution resumed from, plus all volunteer work on tasks that miss their deadline. Each configuration is run with 300 independent seeds; all strategies see the same devices, tasks, and interruption sequence for a given seed. Each experiment uses its own seed range, so the threshold study and the main comparison use disjoint seeds. We report the mean ± the 95% confidence-interval half-width, $1.96\, s/\sqrt{n}$, and call a difference significant only when the two intervals do not overlap, a conservative criterion; where intervals overlap we do not claim a difference.

VI. RESULTS

Wasted CPU and cloud use are reported in thousands of device-seconds and instance-seconds per run (about 152 tasks).

A. Comparison with volunteer-only baselines

Table III and Fig. 2 compare the strategies as designed, with only the proposed scheduler able to use the cloud. "Proposed, tuned" uses the hedging thresholds selected in Section VI-E and is discussed there.

Deadline completion. Under stable conditions all strategies complete about 99.8% of tasks except capability + stability (99.18%), and the proposed scheduler (99.85%) is

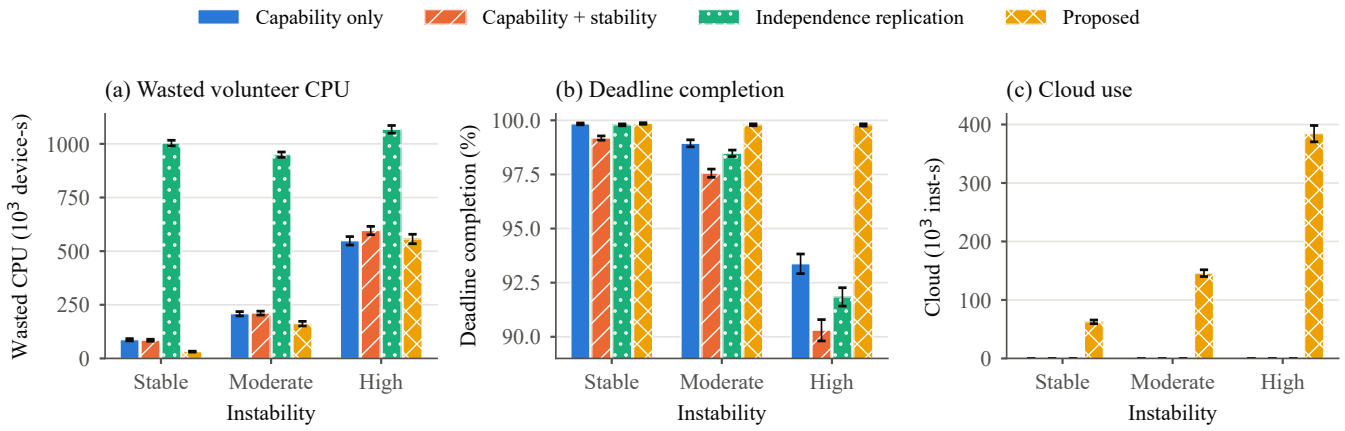


Fig. 2. Wasted volunteer CPU (a), deadline completion (b), and cloud use (c) for each strategy and instability level, with 95% confidence intervals (Table III). Only the proposed scheduler uses the cloud in this comparison. Panel (b) does not start at zero.

indistinguishable from capability only (99.84%). As instability grows, the volunteer-only baselines degrade while the proposed scheduler does not: under moderate instability it completes 99.79% of tasks versus 98.94% for the best baseline, and under high instability 99.79% versus 93.37%, a gap of 6.4 percentage points. Ranking by stability alone is worse than ranking by capability in every condition, because it ignores CPU speed (Section VI-C).

Wasted CPU. Static two-way replication wastes the most volunteer CPU in every condition (0.95–1.07 million device-seconds), because one full copy of every task is always discarded. Under stable conditions the proposed scheduler wastes 64% less than capability only; under moderate instability, 22% less. Under high instability its waste is statistically indistinguishable from capability only and 48% below replication.

Cloud use. The proposed scheduler's reliability is partly bought with cloud capacity: 62.5, 145.8, and 384.3 thousand instance-seconds per run under stable, moderate, and high instability. The next subsection asks whether the baselines would do as well if they had the same access to the cloud.

B. Equal cloud access

Table IV repeats the comparison on the same seeds, now letting each baseline restart a task in the cloud once all of its volunteer copies are lost. Fig. 3 places every strategy on the cloud–waste plane for the high-instability case.

With cloud recovery, capability only becomes a strong baseline: it completes 99.77–99.92% of tasks, matching the proposed scheduler. The difference lies in what each strategy spends. Capability only + cloud restarts every lost task in the cloud from the beginning, whereas the proposed scheduler resumes from checkpoints and uses volunteer hedges first. As a result, the proposed scheduler uses 71%, 65%, and 46% less cloud time under stable, moderate, and high instability. With its default thresholds it pays for this saving in volunteer CPU under moderate and high instability, wasting 27% and 168% more than capability only + cloud, because hedges that are cancelled when the primary finishes are discarded work. Replication + cloud uses the least cloud time under stable and moderate conditions, but under those conditions it wastes 15–38 times more volunteer CPU than the tuned proposed

TABLE IV
BASELINES WITH CLOUD RECOVERY (SAME SEEDS AND UNITS AS TABLE III)

Cond.	Method	Deadline (%)	Wasted CPU	Cloud
Stable	Capability only	99.92 ± 0.03	68.3 ± 3.6	212.8 ± 11.8
	Cap. + stability	99.47 ± 0.07	65.6 ± 3.1	135.9 ± 5.6
	Indep. replication	99.86 ± 0.03	999.1 ± 12.9	28.2 ± 2.4
Moderate	Capability only	99.86 ± 0.04	127.7 ± 4.5	411.4 ± 15.0
	Cap. + stability	98.91 ± 0.10	135.2 ± 4.7	289.6 ± 9.0
	Indep. replication	99.43 ± 0.07	909.4 ± 11.6	126.5 ± 6.1
High	Capability only	99.77 ± 0.05	207.4 ± 4.4	704.7 ± 15.3
	Cap. + stability	98.24 ± 0.14	256.7 ± 6.1	610.2 ± 12.5
	Indep. replication	98.33 ± 0.13	823.4 ± 10.3	422.8 ± 11.4

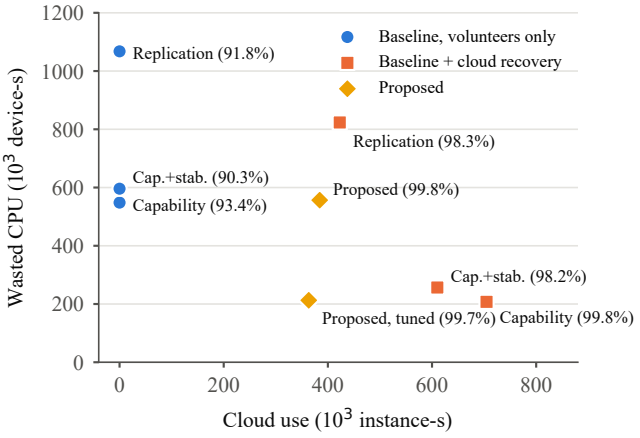


Fig. 3. Cloud use versus wasted volunteer CPU under high instability, with deadline completion in parentheses (Tables III and IV). Lower and further left is better.

TABLE V
ABLATION UNDER HIGH INSTABILITY (30 DEVICES, 300 SEEDS; UNITS AS IN TABLE III)

Configuration	Deadline (%)	Wasted CPU	Cloud
Capability only	93.77 ± 0.46	531.0 ± 21.0	0
+ Stability	90.56 ± 0.51	590.7 ± 20.1	0
+ Conditional survival	93.99 ± 0.43	452.0 ± 18.5	0
+ Dispatch levels	94.32 ± 0.39	695.8 ± 24.2	0
+ Live risk controller	99.55 ± 0.07	493.4 ± 18.3	0
+ Hazard multipliers	99.65 ± 0.07	615.9 ± 19.2	0
+ Co-availability	99.65 ± 0.07	611.5 ± 19.1	0
+ Cloud fallback (full)	99.86 ± 0.04	556.8 ± 20.0	373.0 ± 13.5

scheduler and completes fewer tasks under moderate (99.43%) and high (98.33%) instability.

Because cloud time is the only billed resource in this model, the cloud saving translates directly into cost. Under high instability the tuned proposed scheduler uses about 666 cloud instance-hours per 1,000 tasks, against 1,291 for capability only + cloud; at an illustrative price of $0.10 per instance-hour, that is about $67 instead of $129 per 1,000 tasks. The ratio does not depend on the price chosen.

C. Ablation

Table V adds components one at a time under high instability, on a separate set of 300 seeds.

Ranking by stability alone lowers deadline completion from 93.77% to 90.56%. Stability says nothing about speed: on the first 30 ablation seeds, 19% of tasks placed by stability ran on the slowest (2-core) devices, against 1% under capability ranking, and the mean speed factor of the chosen device fell from 1.44 to 1.17. Tasks therefore run about 20% longer and miss more deadlines. Adding conditional survival restores capability terms through the suitability score.

The live risk controller is the decisive component. Adding it raises deadline completion from 94.32% to 99.55% and lowers wasted CPU by 29%, even without any cloud access. Choosing execution levels only at dispatch time does not help: it starts hedges from scratch for tasks that look risky on arrival, which increases wasted CPU by 54% over ranking by conditional survival alone. The state-dependent hazard multipliers do not improve deadline completion significantly (99.55% to 99.65%, overlapping intervals) and increase wasted CPU by 25%. In our simulator a laptop on battery is interrupted only when its battery runs out, and network fluctuations do not cause interruptions at all, so the multipliers inflate risk estimates and trigger unnecessary hedges. Co-availability-aware hedge selection has no measurable effect at this pool size. Finally, the cloud fallback raises deadline completion to 99.86% and reduces wasted CPU by 9%, at a cost of 373 thousand instance-seconds.

D. Co-availability

In the system-level experiment with 40 devices (Table VI), choosing hedges by co-availability performs the same as choosing them by individual suitability: wasted CPU, deadline completion, and replica-hours all lie within each other's confidence intervals. To isolate the placement decision itself, we sampled 500 pools of 28 devices in four correlated groups, placed three replicas next to a primary using the first half of each device's availability history, and measured the probability that all copies were down at once on the held-out second half (Table VII). Co-availability-aware placement reduces this joint interruption probability from 2.42% to 0.54%, and the share of pools that miss a 2% target from 39.4% to 1.2%. Co-availability thus clearly improves the reliability of a replica set when correlation is strong, but in the full system, where most hedges finish or are cancelled before a correlated outage strikes, this does not translate into a measurable benefit.

TABLE VI
SYSTEM-LEVEL HEDGE SELECTION (40 DEVICES, HIGH INSTABILITY, 300 SEEDS)

Hedge choice	Wasted CPU	Deadline (%)	Replica-h/task
Individual suitability	491.4 ± 18.2	99.82	1.44
Co-availability	502.8 ± 18.5	99.84	1.45

TABLE VII
ISOLATED PLACEMENT OF THREE REPLICAS (500 POOLS, 28 DEVICES, 4 CORRELATED GROUPS; JOINT INTERRUPTION MEASURED ON HELD-OUT HISTORY)

Placement	Replicas	Joint int. (%)	Runs $> 2\%$
Independence	3	2.42	39.4
Co-availability	3	0.54	1.2

E. Threshold sensitivity and the tuned operating point

Table VIII varies the hedging thresholds with $R_1 = 0.05$ under high instability, on a seed set disjoint from the main comparison. Hedging earlier ($R_2 = 0.10$) gives the highest deadline completion (99.93%) but wastes four times as much volunteer CPU as hedging late ($R_2 = 0.40$). Between the default (0.20, 0.50) and the latest setting (0.40, 0.70), deadline completion does not differ significantly, while wasted CPU falls by 61% and replicas per task fall from 1.13 to 0.61.

We therefore selected $(R_2, R_3) = (0.40, 0.70)$ and evaluated it on the main seeds ("Proposed, tuned" in Table III). The result holds out of sample. Compared with the default thresholds, the tuned scheduler reduces wasted CPU by 17%, 64%, and 62% under stable, moderate, and high instability, with no significant change in deadline completion or cloud use. Compared with capability only + cloud, it wastes 62% and 54% less volunteer CPU under stable and moderate instability and the same amount under high instability. Its deadline completion is the same under moderate and high instability and 0.06 points lower under stable conditions, and it uses 48–69% less cloud time. The tuned operating point therefore removes the volunteer-CPU cost seen with the default thresholds.

F. Scaling with pool size

Table IX and Fig. 4 vary the number of volunteer devices under high instability with the default thresholds. Deadline completion rises from 98.19% with 10 devices to about 99.8%

TABLE VIII
SENSITIVITY TO HEDGING THRESHOLDS (HIGH INSTABILITY, $R_1 = 0.05$, 300 SEEDS; UNITS AS IN TABLE III)

R_2	R_3	Deadline (%)	Wasted CPU	Cloud	Repl.
0.10	0.40	99.93 ± 0.02	861.4 ± 18.1	454.3 ± 18.6	1.64
0.20	0.50	99.82 ± 0.04	560.4 ± 20.9	389.6 ± 13.8	1.13
0.30	0.60	99.76 ± 0.05	342.6 ± 17.5	371.2 ± 10.5	0.80
0.40	0.70	99.76 ± 0.05	216.9 ± 12.6	366.1 ± 9.1	0.61

TABLE IX
SCALING WITH POOL SIZE (HIGH INSTABILITY, 300 SEEDS; UNITS AS IN TABLE III)

Devices	Deadline (%)	Wasted CPU	Cloud	Repl.
10	98.19 ± 0.36	583.5 ± 17.8	725.3 ± 23.9	1.29
20	99.70 ± 0.06	607.4 ± 19.1	496.4 ± 17.8	1.27
30	99.81 ± 0.05	553.2 ± 19.2	379.8 ± 13.5	1.13
40	99.80 ± 0.04	489.5 ± 19.4	312.6 ± 9.6	0.99
50	99.81 ± 0.04	430.6 ± 18.2	275.4 ± 7.8	0.86

with 30 or more, while cloud use falls by 62% from 10 to 50 devices and replicas per task fall from 1.29 to 0.86. A larger pool offers more free, capable devices for hedges, so fewer tasks need the cloud.

VII. DISCUSSION AND LIMITATIONS

Three observations stand out. First, re-evaluating risk while tasks run matters far more than improving the dispatch decision: the live risk controller alone accounts for most of the gain in deadline completion. Second, in a hybrid volunteer–cloud system the scheduler's main benefit is economic. Simply restarting lost tasks in the cloud achieves the same deadline completion; the proposed scheduler reaches it with a third to a half of the cloud time, by resuming from checkpoints and hedging on volunteers first. Third, the hedging thresholds set the balance between reliability and wasted volunteer CPU, and a late-hedging setting chosen on separate seeds kept reliability while removing most of the wasted work.

The results have the following limitations.

- *Synthetic environment.* Availability, correlated outages, and power behaviour come from a synthetic model with exponential up and down times, not from real volunteer traces, and the pools are small (at most 50 devices).
- *Risk-model mismatch.* The risk model assumes that hazard grows with session age and that battery and network state raise it, but the simulated interruptions are memoryless and are not caused by network state. The ablation shows that the multipliers hurt in this setting; they should be calibrated on real traces before use.
- *Tuning.* The thresholds were selected on a separate seed set of the same simulator, which guards against overfitting to particular seeds but not to the simulator itself.
- *Simplified resources.* The cloud has unlimited capacity and starts instantly apart from checkpoint transfer; each volunteer runs one replica at a time; checkpoint size and transfer time are model parameters; and the cost figure uses an illustrative price.
- *Scope.* Tasks are independent; DAG workflows and result verification are not modelled.

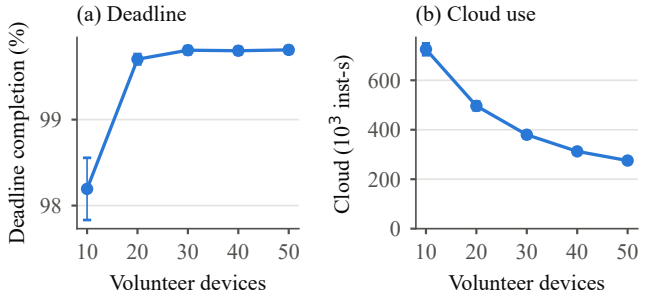


Fig. 4. Effect of pool size under high instability: deadline completion (a) and cloud use (b), with 95% confidence intervals (Table IX).

The reported values should therefore be read as comparisons between strategies within one simulator, not as predictions of production performance.

VIII. CONCLUSION AND FUTURE WORK

We presented a stability-aware scheduler for volunteer computing on heterogeneous desktops and laptops that monitors the interruption risk of running tasks, adds checkpoint-seeded volunteer hedges when risk rises, and falls back to the cloud when it is high. In simulation with 300 seeds per configuration, the scheduler keeps deadline completion near 99.8% under high instability, where volunteer-only baselines fall to 90.3–93.4%. Against a baseline given the same cloud access, it achieves the same deadline completion with 46–71% less cloud time. With hedging thresholds chosen on separate seeds, it also wastes no more volunteer CPU than that baseline. The live risk controller is the most important component, while the state-dependent hazard multipliers did not help in our environment. Overall, the results indicate that reliability-aware scheduling should be a continuous process rather than a single decision made at dispatch, and that in hybrid volunteer–cloud systems its main payoff is a lower cloud bill.

Future work will (i) evaluate the scheduler on real volunteer availability traces and physical devices; (ii) calibrate the hazard model and its multipliers from those traces; (iii) model cloud start-up delay, capacity limits, and real pricing; (iv) extend the approach to workflow (DAG) scheduling; and (v) adapt the hedging thresholds online.

CODE AVAILABILITY

The simulator, all experiment configurations, and the scripts that regenerate every table and figure are available at https://github.com/71percentbanana/Cloud-project.

REFERENCES

[1] D. P. Anderson, "BOINC: A platform for volunteer computing," *Journal of Grid Computing*, vol. 18, no. 1, pp. 99–122, 2020, doi: 10.1007/s10723-019-09497-9.

[2] M. I. Zimmerman, J. R. Porter, M. D. Ward, S. Singh *et al.*, "SARS-CoV-2 simulations go exascale to predict dramatic spike opening and cryptic pockets across the proteome," *Nature Chemistry*, vol. 13, no. 7, pp. 651–659, 2021, doi: 10.1038/s41557-021-00707-0.

[3] D. P. Anderson, "Globally scheduling volunteer computing," *Future Internet*, vol. 13, no. 9, p. 229, 2021, doi: 10.3390/fi13090229.

[4] S. Gonzalo San José, J. M. Marquès, A. García-Villoria, J. Panadero, and L. Calvet, "CLARA: A novel clustering-based resource-allocation mechanism for exploiting low-availability complementarities of voluntarily contributed nodes," *Future Generation Computer Systems*, vol. 128, pp. 248–264, 2022, doi: 10.1016/j.future.2021.10.002.

[5] S. Gonzalo San José, J. M. Marquès, J. Panadero, and L. Calvet, "NARA: Network-aware resource allocation mechanism for minimizing quality-of-service impact while dealing with energy consumption in volunteer networks," *Future Generation Computer Systems*, vol. 164, p. 107593, 2025, doi: 10.1016/j.future.2024.107593.

[6] A. Waheed, M. A. Shah, A. Khan *et al.*, "An infrastructure-assisted job scheduling and task coordination in volunteer computing-based VANET," *Complex & Intelligent Systems*, vol. 9, no. 4, pp. 3613–3633, 2023, doi: 10.1007/s40747-021-00437-3.

[7] P. Ma, S. Garg, and M. Barika, "Research allocation in mobile volunteer computing system: Taxonomy, challenges and future work," *Future Generation Computer Systems*, vol. 154, pp. 251–265, 2024, doi: 10.1016/j.future.2024.01.015.

[8] A. Pandey, P. Calyam, S. Debroy, S. Wang, and M. L. Alarcon, "VECTrust: Trusted resource allocation in volunteer edge-cloud computing workflows," in *Proc. 14th IEEE/ACM International Conference on Utility and Cloud Computing (UCC)*, 2021, pp. 1–10, doi: 10.1145/3468737.3494099.

[9] X. Cao, C. Chen, S. Li, C. Lv, J. Li *et al.*, "Research on computing task scheduling method for distributed heterogeneous parallel systems," *Scientific Reports*, vol. 15, p. 8937, 2025, doi: 10.1038/s41598-025-94068-0.

[10] M. Malekimajd and A. Safarpoor-Dehkordi, "A survey on cloud computing scheduling algorithms," *Multiagent and Grid Systems*, vol. 18, no. 2, pp. 119–148, 2022, doi: 10.3233/MGS-220217.

[11] M. A. Shahid, N. Islam, M. M. Alam, M. S. Mazliham, and S. Musa, "Towards resilient method: An exhaustive survey of fault tolerance methods in the cloud computing environment," *Computer Science Review*, vol. 40, p. 100398, 2021, doi: 10.1016/j.cosrev.2021.100398.

[12] J. Jia, Y. Liu, G. Zhang, Y. Gao, and D. Qian, "Software approaches for resilience of high performance computing systems: a survey," *Frontiers of Computer Science*, vol. 17, no. 4, p. 174105, 2023, doi: 10.1007/s11704-022-2096-3.

[13] M. Farid, O. A. Amodu, H. S. Lim, J. A. J. Alsayaydeh, M. F. Abdullah, and F. A. Saif, "A survey of fault and intrusion tolerance approaches for scientific workflow scheduling in cloud computing," *Computers*, vol. 15, no. 5, p. 304, 2026, doi: 10.3390/computers15050304.

[14] S. Bharany, S. Badotra, S. Sharma, S. Rani, M. Alazab, R. H. Jhaveri, and T. R. Gadekallu, "Energy efficient fault tolerance techniques in green cloud computing: A systematic survey and taxonomy," *Sustainable Energy Technologies and Assessments*, vol. 53, p. 102613, 2022, doi: 10.1016/j.seta.2022.102613.

[15] P. Kumari and P. Kaur, "An adaptable approach to fault tolerance in cloud computing," *International Journal of Performability Engineering*, vol. 19, no. 1, pp. 43–54, 2023, doi: 10.23940/ijpe.23.01.p5.4354.

[16] A. R. Setlur, S. J. Nirmala, H. S. Singh, and S. Khoriya, "An efficient fault tolerant workflow scheduling approach using replication heuristics and checkpointing in the cloud," *Journal of Parallel and Distributed Computing*, vol. 136, pp. 14–28, 2020, doi: 10.1016/j.jpdc.2019.09.004.

[17] M. Khaleel, "Failure-aware resource provisioning for hybrid computation offloading in cloud-assisted edge computing using gravity reference approach," *Swarm and Evolutionary Computation*, vol. 91, p. 101704, 2024, doi: 10.1016/j.swevo.2024.101704.

[18] S. Tomar, A. K. Mishra, and D. K. Yadav, "Knowledge-based checkpointing strategy for spot instances in cloud computing," *Journal of Current Science and Technology*, vol. 13, no. 2, pp. 412–427, 2023, doi: 10.59796/jcst.V13N2.2023.1754.

[19] L. Teylo, L. Arantes, P. Sens, and L. M. A. Drummond, "A dynamic task scheduler tolerant to multiple hibernations in cloud environments," *Cluster Computing*, vol. 24, no. 2, pp. 1051–1073, 2021, doi: 10.1007/s10586-020-03175-2.

[20] S. S. Gill, X. Ouyang, and P. Garraghan, "Tails in the cloud: a survey and taxonomy of straggler management within large-scale cloud data centres," *The Journal of Supercomputing*, vol. 76, no. 12, pp. 10 050–10 089, 2020, doi: 10.1007/s11227-020-03241-x.

[21] S. Tuli, S. S. Gill, P. Garraghan, R. Buyya, G. Casale, and N. R. Jennings, "START: Straggler prediction and mitigation for cloud computing environments using encoder LSTM networks," *IEEE Transactions on Services Computing*, vol. 16, no. 1, pp. 615–627, 2023, doi: 10.1109/TSC.2021.3129897.

[22] M. S. Jassas and Q. H. Mahmoud, "Analysis of job failure and prediction model for cloud computing using machine learning," *Sensors*, vol. 22, no. 5, p. 2035, 2022, doi: 10.3390/s22052035.

[23] T. N. T. Asmawi, A. Ismail, and J. Shen, "Cloud failure prediction based on traditional machine learning and deep learning," *Journal of Cloud Computing*, vol. 11, p. 47, 2022, doi: 10.1186/s13677-022-00327-0.

[24] M. Gollapalli *et al.*, "Task failure prediction using machine learning techniques in the Google cluster trace cloud computing environment," *Mathematical Modelling of Engineering Problems*, vol. 9, no. 2, pp. 545–553, 2022, doi: 10.18280/mmep.090234.

[25] M. Ma, Y. Liu, Y. Tong, H. Li, P. Zhao, Y. Xu, H. Zhang, S. He, L. Wang, Y. Dang, S. Rajmohan, and Q. Lin, "An empirical investigation of missing data handling in cloud node failure prediction," in *Proc. 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2022, pp. 1453–1464, doi: 10.1145/3540250.3558946.

[26] D. Papathanasiou, K. Demertzis, and N. Tziritas, "Machine failure prediction using survival analysis," *Future Internet*, vol. 15, no. 5, p. 153, 2023, doi: 10.3390/fi15050153.

[27] K. A. Mills, R. Chandrasekaran, and N. Mittal, "Algorithms for optimal replica placement under correlated failure in hierarchical failure domains," *Theoretical Computer Science*, vol. 809, pp. 482–518, 2020, doi: 10.1016/j.tcs.2020.01.004.